SymComb: Visual Analytics for Multi-Tuner Comparison in Symbolic Execution Tuning

Category: Research

Abstract

Symbolic execution engines such as KLEE depend on numerous tunable parameters that affect branch coverage, and both domain-specific and general-purpose tuners have been proposed to automate their configuration. In practice, researchers apply multiple tuners to the same target program and compare their results, yet existing tools summarize each tuner's performance as a scalar metric and do not reveal where each tuner searches, how the resulting test cases overlap, or whether combining tuners can improve coverage. We present *SymComb*, a visual analytics system that spatializes Multi-Tuner Search Logs produced by multiple tuners over a hexagonal partition of the parameter space. *SymComb* summarizes each cell by its tuner composition and coverage outcome, and identifies cells whose test cases add the most new branches relative to a selected anchor cell. We demonstrate *SymComb* through a use case on three target programs from the KLEE benchmark with six tuners.

Index Terms: Radiosity, global illumination, constant time.

1 Introduction

In software testing, symbolic execution [9, 5, 15, 4] is a method that automatically generates test cases for a program by systematically exploring its execution paths. The behavior and effectiveness of symbolic execution engines depend on numerous tunable parameters, such as search strategies and constraint-solving options [6, 10, 13]. Tuning these parameters is known to be a critical challenge [3]. To automate this process, researchers have proposed both general-purpose black-box optimization algorithmsTODO: cite cma-es, tpe, bo and domain-specific tuners [7, 6, 10] that account for mixed parameter types and non-linear relationships between parameters and branch coverage. These tuners employ different search strategies and may concentrate on different regions of the parameter space. For example, one tuner may broadly sample the space and achieve high overall coverage, while another may focus on a narrow region and cover branches that the first misses. Combining their results can therefore yield higher cumulative coverage, and as the number of available tuners grows, researchers need tools to compare tuner exploration and locate such complementary opportunitiesTODO: cite SE tuner comparison evidence.

A tuning session produces Multi-Tuner Search Logs, where each trial carries four elements: a parameter configuration, a branch coverage value, a branch coverage vector indicating which branches were executed, and the identity of the tuner that generated it. Together, these elements describe both where each tuner searched and how the resulting test cases overlap. However, existing tools expose little of this structure. Current practice in symbolic execution summarizes each tuner's performance as a single branch coverage value [7, 6]. Such aggregates hide where each tuner concentrated its search, which parameter values led to coverage differences, and whether results from different tuners are complementary. Visual analytics systems for hyperparameter optimization [14, 16, 8] focus on a single optimizer and likewise reduce performance to a scalar metric. A prior visual analytics approach for symbolic execution tuning [11] supports comparison between user-defined trial groups within a single experiment. Yet it does not compare exploration patterns across multiple tuners, does not relate individual parameter values to each tuner's behavior, and requires multi-step manual analysis to identify complementary regions.

To address this gap, we present *SymComb*, a visual analytics system for comparing the trials produced by multiple tuners on a symbolic execution engine and identifying complementary cells across them. *SymComb* spatializes Multi-Tuner Search Logs by partitioning the parameter space into hexagonal cells, encodes each cell by its tuner composition and coverage outcome, and identifies cells whose test cases add the most new branches relative to a selected anchor cell. We demonstrate *SymComb* through a use case on three target programs from the KLEE benchmark with six tuners that include both domain-specific and general-purpose optimization algorithms. In summary, this paper makes the following contributions:

- a **problem formulation** of multi-tuner search log analysis as a joint inspection of parameter-space exploration and branch-set overlap, captured by the Multi-Tuner Search Logs abstraction,
- a **method** that combines a shared hexagonal partition of the parameter space with anchor-based identification of complementary cells, and
- a **use case** on three target programs with six tuners, in which *SymComb* surfaces tuner-specific exploration patterns and complementary cells that add new branches beyond a chosen anchor.

2 Related Work

2.1 Visual Analytics for Parameter Tuning

Visual analytics systems for (hyper)parameter tuning have developed along two lines in the broader optimization literature. General HPO systems [14, 16, 1] visualize configurations from a single optimizer using parallel coordinates or projected scatterplots, and map each configuration to a scalar performance metric. Systems for multi-algorithm comparison in evolutionary or multi-objective optimization [8, 12] juxtapose runs from different algorithms and contrast them through aggregate quality indicators such as hypervolume, but typically do not prioritize the parameter space itself.

In symbolic execution, visual analytics support is limited to two systems. A prior approach for parameter tuning [11] provides overviews of parameters and coverage, and supports comparison between user-defined trial groups within a single tuner. Complementarity between such groups is defined and can be computed, but identifying complementary groups requires multi-step manual analysis. SymNav [2] targets execution traces rather than parameter tuning.

In contrast, *SymComb* spatializes the parameter space through a hexagonal partition shared by multiple tuners, and automatically identifies complementary cells with respect to an anchor chosen by the analyst. In the remainder of this section, we situate this choice within prior work on comparative and spatial visualization.

2.2 Comparative and Spatial Visualization

Comparing multiple related datasets or model runs is a common task in visual analytics, and three general strategies (juxtaposition, superposition, and explicit encoding of differences) have been characterizedTODO: cite gleicher2011. Juxtaposition may become unwieldy as the number of compared items grows, superposition requires a shared reference frame, and explicit encoding requires a

prior decision on what difference to encode. For comparing exploration patterns across many tuners, a spatial layout that embeds configurations into a shared plane offers a flexible starting point, in which each tuner's trials can be overlaid, contrasted, or aggregated after the layout is fixed.

Spatial layouts of non-spatial data have been explored in several forms, including tilemaps for geographic summariesTODO: cite tilemap and gridded glyphmaps for spatial modelingTODO: cite glyphmap. These designs aggregate multivariate data over a regular partition, trading spatial precision for legibility of summary glyphs. *SymComb* extends this idea to parameter configurations, which lack an inherent spatial embedding. We apply dimensionality reduction to obtain a 2D layout, partition it into hexagonal cells, and aggregate trials within each cell, so that exploration patterns and performance differences across tuners can be read at a glance.

3 Multi-Tuner Search Logs

We refer to the data produced by running multiple tuners on a symbolic execution engine as Multi-Tuner Search Logs. Each tuning session generates a set of trials, where each trial carries four elements: (i) a *parameter configuration*, a vector of values for the engine's tunable parameters, (ii) a *branch coverage value*, the number of branches executed by the resulting test cases, (iii) a *branch coverage vector*, the set of branch indices that were executed, and (iv) a *tuner identity*, indicating which tuner generated the trial. Together, these elements describe both *where* each tuner searched in the parameter space and *which branches* its trials covered.

Dataset characteristics. The instance of Multi-Tuner Search Logs that we use in this work is produced by running six tuners—SymTunerTODO: cite, ParaDySETODO: cite, LearchTODO: cite, and three general-purpose black-box optimizers (CMA-ESTODO: cite, TPETODO: cite, and Bayesian optimizationTODO: cite)—on three target programs: `gawk`, `gcal`, and `grep`. Each tuner runs 2,200 trials per program, yielding 13,200 trials per program and 39,600 trials overall. The symbolic execution engine (KLEE) exposes 61 parameters, of which 35 are boolean, 5 are categorical, and 21 are numeric. Branch counts range from roughly 3,400 (`grep`) to 4,500 (`gcal`).

Analytical tasks. From discussions with researchers working on symbolic execution tuning and our review of how tuners are typically compared, we identify three tasks that Multi-Tuner Search Logs can support.

- **T1. Tuner-centric exploration and performance comparison.** Understanding how each tuner explores the parameter space and how its exploration translates into branch coverage, in order to identify which tuner is most effective in which region.
- **T2. Contrastive inspection of parameter values.** Examining how regions associated with particular values of a parameter differ in tuner composition and coverage, in order to relate specific parameter choices to tuner behavior.
- **T3. Identification of complementary combinations.** Finding pairs or groups of regions whose trials, when merged, increase the accumulated branch coverage, in order to construct complementary combinations across tuners.

4 *SymComb*

We designed *SymComb* around three design goals that address the analytical tasks identified in Section 3.

G1. Shared spatial reference for multi-tuner comparison. To support T1 and T2, trials from different tuners must be placed into a common frame of reference where exploration patterns and coverage outcomes can be read side by side. A per-tuner juxtaposition of views would scale poorly with the number of tuners and would obscure regions where tuners overlap. We therefore embed all trials into a single hex map whose layout is computed from parameter similarity alone, independent of tuner identity.

G2. Automatic identification of complementary regions. To support T3, the system should reduce the manual effort involved in finding complementary regions. Rather than asking the analyst to define candidate groups and compute complementarity by hand, *SymComb* takes an *anchor* cell selected by the analyst and scores every other cell automatically, surfacing the most complementary candidates on the map.

G3. Interactive deployment without a backend. Multi-Tuner Search Logs, together with intermediate artifacts such as cluster centroids and distance matrices, is heavy enough to make in-browser recomputation impractical. *SymComb* precomputes the expensive pipeline steps offline and serves only static JSON to the client, keeping interactions responsive while avoiding server-side infrastructure.

4.1 Hex Map Encoding

The central view of *SymComb* is a hex map in which each cell summarizes a cluster of trials with similar parameter values (Figure **??**).

From trials to cells. All trials from the selected tuners are pooled into a single collection, giving tuners a shared reference frame (G1). We encode each parameter configuration as a fixed-length vector with a mixed-type encoding (normalized numeric, binary, and one-hot categorical) and apply k-means clustering. *SymComb* supports five resolution levels with $k \in \{12, 24, 50, 100, 200\}$; the finest level runs k-means on the raw trials, and coarser levels are obtained by hierarchical merging. To place clusters on a plane, we compute a distance matrix between cluster centroids and apply classical multidimensional scaling (MDS), which preserves centroid-level distances rather than local neighborhood structure. Per-parameter distances are combined via $\sqrt{\sum_i d_i^2 / n_p}$, where n_p is the number of parameters. For categorical parameters, we use total variation distance between centroid distributions to prevent categories with many values from dominating. The 2D coordinates are snapped to a hexagonal grid in axial coordinates, with collisions resolved by a spiral search so that every cluster occupies exactly one cell. The hex size is set adaptively as $32 \cdot \sqrt{200/k}$ pixels. Neighboring cells therefore tend to contain similar parameter configurations, and the layout remains fixed as the analyst toggles tuners or changes the visual encoding.

Two-level hex design. Each cell uses a two-level hex: an outer hex conveying the tuner composition, and an inner hex, drawn at 45% of the outer size, conveying either a coverage outcome or a parameter value (Figure **??**a). This separation lets analysts read tuner-level and performance-level information from a single cell without overloading either channel.

Tuner composition (T1). Each tuner is assigned a distinct hue. We color the outer hex of a cell by the *dominant tuner*, defined as the tuner with the most trials in the cluster, and blend the hue with 55% white to keep the outer ring visually recessive. Encoding only the dominant tuner trades cell-level detail for map-level legibility: a pie chart or stacked bar per cell would scale poorly to 200 cells and six tuners. The full tuner distribution of a cell is available on demand through a tooltip, and analysts can filter tuners via the legend, which recomputes all cell-level statistics over the selected subset.

Coverage outcome (T1, T3). The inner hex is colored on a sequential YlOrBr scale according to a user-selectable coverage metric: mean, maximum, minimum, mean marginal, or cumulative coverage of the cell's trials. The mean option is primarily used to read where each tuner performs well (T1), while the cumulative option prepares the map for identifying complementary regions (T3). The color scale is normalized to the global range of the chosen metric across all cells. As an additional interaction, the analyst may re-encode the inner hex to show the binned value of a selected param-

eter (e.g., *Low*/*Mid*/*High* for numeric parameters), which supports contrastive inspection of parameter values across cells.

4.2 Complementary Recommendation

To support T3 (G2), *SymComb* computes complementarity scores across cells and surfaces cells that add the most branches relative to an anchor chosen by the analyst. A prior visual analytics system for symbolic execution tuning [11] defines the complementarity of two trial groups g_1, g_2 as $\text{accum}(g_1 \cup g_2) - \max(\text{accum}(g_1), \text{accum}(g_2))$, where $\text{accum}(\cdot)$ is the size of the union of branch coverage vectors over a group. *SymComb* reuses this measure but restructures its use around an *anchor* cell chosen by the analyst. Given an anchor cell a, the complementarity of any other cell c is

$$\text{com}(a,c) = |B(c) \setminus B(a)|, \tag{1}$$

where $B(\cdot)$ is the branch coverage vector of a cell viewed as a set. This is equivalent to the prior-work definition when the anchor covers at least as many branches as the other cell, and it admits an efficient single pass over every cell once the anchor is selected.

On selecting an anchor, *SymComb* computes $\text{com}(a,c)$ for every cell at the current resolution and recolors the map on a green intensity scale, with the top five candidates highlighted by a contrasting border (Figure **??**b). Combined with tuner filtering, the mode lets analysts ask questions such as "*among the cells dominated by tuners other than the one I chose, which would add the most new branches to my anchor?*" and read the answer directly from the map.

4.3 Implementation

SymComb is implemented as a React-based web application with D3.js for hex rendering. Data preparation, including clustering, MDS, and the $B(\cdot)$ sets used by the complementarity computation, is performed offline in Python and exported as static JSON, which the client loads once per program (G3). All randomized steps in the pipeline use fixed seeds, so the same Multi-Tuner Search Logs always produces the same hex layout. For the dataset described in Section 3, the precomputed artifact is roughly 40 MB per target program and the complementary recommendation over 200 cells completes in under a second in the browser.

5 Use Case: Comparing and Combining Tuners on `gawk`

We illustrate how *SymComb* supports the three tasks through an analysis of the `gawk` program, where all six tuners were run for 2,200 trials each.

T1: Where does each tuner explore? [Figure 1(a) 참조하며 관찰된 패턴 서술] [예: "SymTuner's trials cluster in a region of the map dominated by high values of parameter X, while TPE spreads more evenly..."]

T2: Which parameters drive these differences? [Parameter overlay를 적용했을 때의 발견] [예: "Selecting parameter Y reveals that the region dominated by SymTuner corresponds to High values of Y..."]

T3: Which tuners complement each other? [Complementary mode에서 anchor 선택 후 발견한 상보 조합] [Figure 1(b) 참조] [예: "Selecting the best SymTuner cell as anchor, the top five complementary cells are all dominated by CMA-ES and TPE, adding N new branches..."]

6 Future Work and Conclusion

We presented *SymComb*, a visual analytics system for comparing and combining the trials produced by multiple tuners on a symbolic execution engine. Through its hexagonal partition of the parameter space and anchor-based complementarity recommendation, *SymComb* supports the three tasks of tuner comparison, parameter-level contrast, and complementary combination identification that Multi-Tuner Search Logs make possible.

Generalization to other tuning domains. While we developed *SymComb* for symbolic execution, the four-tuple structure of Multi-Tuner Search Logs has a direct counterpart in hyperparameter optimization for machine learning, where a trial produces a configuration, an aggregate performance metric, a per-instance outcome vector (e.g., per-sample correctness), and an optimizer identity. Prior work on ensemble construction exploits exactly this kind of per-instance diversity between learnersTODO: cite kuncheva ensemble diversity, suggesting that the complementarity-based recommendation in *SymComb* could inform ensemble selection.

Limitations. *SymComb* currently relies on a fixed set of resolution levels and a greedy hex assignment, which may produce artifacts when cluster density varies sharply. The complementarity recommendation is anchor-based, so it depends on the analyst's choice of anchor; finding globally optimal combinations across all cells remains future work. A formal user study with symbolic execution researchers is also needed to assess whether the observed patterns in our use case translate into practical tuning decisions.

References

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '19, p. 2623–2631. Association for Computing Machinery, New York, NY, USA, 2019. doi: 10.1145/3292500.3330701 1

[2] M. Angelini, G. Blasilli, L. Borzacchiello, E. Coppa, D. C. D'Elia, C. Demetrescu, S. Lenti, S. Nicchi, and G. Santucci. Symnav: Visually assisting symbolic execution. In *2019 IEEE Symposium on Visualization for Cyber Security (VizSec)*, pp. 1–11, 2019. doi: 10.1109/VizSec48167.2019.9161524 1

[3] R. Baldoni, E. Coppa, D. C. D'elia, C. Demetrescu, and I. Finocchi. A survey of symbolic execution techniques. *ACM Computing Surveys*, 51(3), May 2018. doi: 10.1145/3182657 1

[4] C. Cadar, D. Dunbar, and D. Engler. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, p. 209–224. USENIX Association, USA, 2008. 1

[5] C. Cadar, V. Ganesh, P. M. Pawlowski, D. L. Dill, and D. R. Engler. Exe: Automatically generating inputs of death. *ACM Transactions on Information and System Security*, 12(2), Dec. 2008. doi: 10.1145/1455518.1455522 1

[6] S. Cha, S. Hong, J. Bak, J. Kim, J. Lee, and H. Oh. Enhancing dynamic symbolic execution by automatically learning search heuristics. *IEEE Transactions on Software Engineering*, 48(9):3640–3663, 2022. doi: 10.1109/TSE.2021.3101870 1

[7] S. Cha, M. Lee, S. Lee, and H. Oh. Symtuner: maximizing the power of symbolic execution by adaptively tuning external parameters. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, p. 2068–2079. Association for Computing Machinery, New York, NY, USA, 2022. doi: 10.1145/3510003.3510185 1

[8] A. Chatzimparmpas, R. M. Martins, K. Kucher, and A. Kerren. Visevol: Visual analytics to support hyperparameter search through evolutionary optimization. *Computer Graphics Forum*, 40(3):201–214, 2021. doi: 10.1111/cgf.14300 1

[9] P. Godefroid, N. Klarlund, and K. Sen. Dart: directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, p. 213–223. Association for Computing Machinery, New York, NY, USA, 2005. doi: 10.1145/1065010.1065036 1

[10] J. He, G. Sivanrupan, P. Tsankov, and M. Vechev. Learning to explore paths for symbolic execution. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Secu-*

rity, CCS '21, p. 2526–2540. Association for Computing Machinery, New York, NY, USA, 2021. doi: 10.1145/3460120.3484813 1

[11] D. Hong, M. Kim, S. Cha, and J. Jo. Symetra: Visual analytics for the parameter tuning process of symbolic execution engines. *arXiv preprint arXiv:2604.05349*, 2026. 1, 3

[12] Y. Huang, Z. Zhang, A. Jiao, Y. Ma, and R. Cheng. A comparative visual analytics framework for evaluating evolutionary processes in multi-objective optimization. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):661–671, Jan. 2024. doi: 10.1109/TVCG.2023.3326921 1

[13] T. Kapus, F. Busse, and C. Cadar. Pending constraints in symbolic execution for better exploration and seeding. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, ASE '20, p. 115–126. Association for Computing Machinery, New York, NY, USA, 2021. doi: 10.1145/3324884.3416589 1

[14] H. Park, Y. Nam, J.-H. Kim, and J. Choo. Hypertendril: Visual analytics for user-driven hyperparameter optimization of deep neural networks. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1407–1416, 2021. doi: 10.1109/TVCG.2020.3030380 1

[15] K. Sen, D. Marinov, and G. Agha. Cute: a concolic unit testing engine for c. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ESEC/FSE-13, p. 263–272. Association for Computing Machinery, New York, NY, USA, 2005. doi: 10.1145/1081706.1081750 1

[16] Q. Wang, Y. Ming, Z. Jin, Q. Shen, D. Liu, M. J. Smith, K. Veeramachaneni, and H. Qu. Atmseer: Increasing transparency and controllability in automated machine learning. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, p. 1–12. Association for Computing Machinery, New York, NY, USA, 2019. doi: 10.1145/3290605.3300911 1